Question 1: Understanding the Algorithm

1. Draw a table with 5 columns (cycle number, value of A_reg, value of B_reg, A_next, B_next) and fill in all of the rows for the first test vector (GCD of 27 and 15). Count the cycle number from 0 when `operands_rdy` and `operands_val` are 1. Fill in the table until the first test vector is done and upload a screenshot of the table. Use decimal numbers instead of binary or hex. Hint: It might be easier to view the waveforms instead of tracing the code. Hint: Take a look starting at 140ns. The signals will be under `gcd_datapath.sv` in the Verdi hierarchy.

	A	B	C	D	E
1	Cycle number	A_reg	B_reg	A_next	B_next
2	0	0	0	27	15
3	1	27	15	12	15
4	2	12	15	15	12
5	3	15	12	3	15
6	4	3	12	12	3
7	5	15	3	9	15
8	6	9	3	6	15
9	7	6	3	3	15
10	8	3	3	0	15
11	9	0	3	3	0
12	10	3	0	27	15
13	11	3	0	27	15

2. In `src/gcd_testbench.sv`, the inputs are changed on the negative edge of the clock. Is the output checked on the positive edge of the clock or the negative edge of the clock? Why?
   The output is checked on the positive edge of the clock. The input are changed on the negedge so that the output can stay stable on the rising clock edge.

3. In `src/gcd_testbench.sv`, what will happen if you change `result_rdy = 1;` to `result_rdy = 0;`? What state will the `gcd_control.sv` state machine be in?

   If I change the result_rdy to 0, there will never be a handshake in the testbench, so there will never be an output and the testbench get stuck in this handshake loop.

By checking the gcd_control.sv, from the code here:

```
if (result_rdy == 1'b1) begin

        nextstate = IDLE;

// if not, stay in this state until the outside is ready for the result
```

the state machine will be in the DONE state.

Question 2: GCD Reports Questions

1. Which report would you look at to find the total number of each different standard cell that the design contains? Hint: Take a look in `build/syn-rundir/reports`.

   final_gates.rpt
2. Which report contains the area breakdown by modules in the design?

   final_area.rpt

3. What is the cell used for `A_register/q_reg[7]`? How much leakage power does `A_register/q_reg[7]` contribute? How did you find this? (*Hint: Take a look at `lab3/build/syn-rundir/gcd.mapped.sdf`*)

   A_register is the register storing operand A. g_reg[7] is the bit 7 of this register
   In the SKY130 lib, under the cell "sky130_fd_sc_hd__dfxtp_1", I find this code:
   cell_leakage_power : 0.0084386350;
   so the cell leakage power is 0.00844nW = 8.44pW

Question 3: GCD Synthesis Questions

1. Considering only the synthesizable SystemVerilog RTL (i.e., excluding testbenches), does the number of synthesized sequential elements match the number of bits of state explicitly instantiated in the design? Explain your answer. (*Hint: How many sequential cells (DFFX1) do we have? What do they correspond to in our design?*)

   From the yosys report, "35 700.672 sky130_fd_sc_hd__dfxtp_1" there are 35 D flip-flops, but it is supposed to be 34 (16 bits in A_reg, 16 bits in B_reg and 2 FSM states).
   It seems yosys changes state encoding from 2-bit binary to one-hot which takes 3-bit width

report from yosys:
7.9.6. Executing FSM_RECODE pass (re-assigning FSM state encoding).
Recoding FSM `$fsm$\GCDctrl0.state_machine.q$192' from module `\gcd' using `auto' encoding:
mapping auto encoding to `one-hot` for this FSM.

Therefore, if I'm using Cadance Genus, the number of synthesized sequential elements should match the number of bits of state

2. Reduce the clock period (in `design.yml`) by the amount of slack (to the nearest nanosecond) in the timing report. Now run the synthesis flow again. Does it still meet timing? Why or why not? Does the critical path stay the same? If not, what changed?
The original period is 20ns and the slack time in the critical path is 14.05ns. Therefore, I set the period 14 ns lower to 6ns. The slack time to the new period is pretty tight – it is only 0.05 ns. OpenSTA doesn't say if it pass or not, but since the slack time is positive, I assume it can pass. However, I am not sure if Genus would fail such a tight slack time.

Question 4: Delay Questions

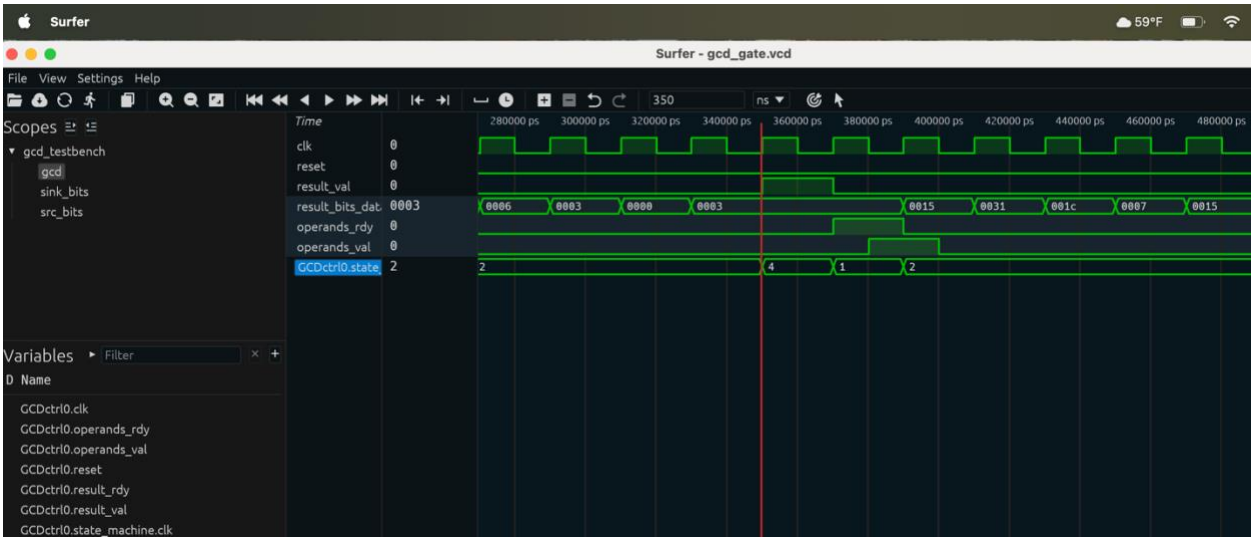
Check the waveforms in Verdi:

```
cd build/sim-rundir
verdi -dbdir simv.daidir -ssf vcdplus.fsdb
```

1. Report the clk-q delay of `state[0]` in `GCDctrl0` at 350 ns and submit a screenshot of the waveforms showing how you found this delay.

   GCDctrl0.state_machine.q[2:0] at 350 ns is 4, so state[0] in GCDctrl0 at 350 ns is 0. State[0] changes from 0 in the previous state to 0 at 350ns, so the delay may not

shows



2. Which line in the SDF file specifies this delay, and what is the specified delay?
   By using OpenSTA, STA timing report tells that the delay is 0.31ns

   0.00 0.00 ^ _472_/CLK (sky130_fd_sc_hd__dfxtp_1)

   0.31 0.31 v _472_/Q (sky130_fd_sc_hd__dfxtp_1)

3. Is the delay observed in the waveform the same as that in the SDF file? Why or why not?
   seems not. I don't see such delay in the waveform. The state changes immediately at 350 ns



Question 5: Synthesized Divider

1. From the reports of your 4-bit synthesized divider, determine its:
    - critical path and slack
      There are 3 critical paths: 171 flip-flop to 175, 171 to 176 and 171 to 177.
      All 3 shares the slack of 17.21ns while the clk period in sta report is 20ns
    - total cell area
      Chip area for module '\divider': 1287.484800 um^2
    - maximum operating frequency in MHz from the reports (You might need to re-run synthesis multiple times to determine the maximum achievable frequency)
      Got this evidence in synthesis report:
      ===== MINIMUM CLOCK PERIOD =====
      clk period_min = 2.79 fmax = 358.16
      So, the max frequency is 358.16 MHz
2. From the reports of your 32-bit synthesized divider, determine its:
    - critical path and slack
      There are 4 critical paths in total, and all share the same slack.
      1. 1486 flip-flop to 1485
      2. 1486 to 1486
      3. 1486 to 1487
      4. 1486 to 1488
      Slack: 10.03 ns while the clk period in sta report is 20ns
    - total cell area
      Chip area for module '\divider': 10088.425600 um^2
    - maximum operating frequency in MHz from the reports (You might need to re-run synthesis multiple times to determine the maximum achievable frequency)
      max f is 100.33
3. Submit your divider code and testbench to Gradescope. Also, `git push` all your work to your GitHub repository. Add comments to explain your testbench and why it provides sufficient coverage for your divider module. (You don't have to run a post-synthesis simulation for Question 5). That is, run `make sim-rtl SIM_RTL_CONF=sim-divider.yml` to verify your testbench.